Algorytmy Równoległe - Laboratorium 5

Adam Staniszewski

22 listopada 2024

Spis treści

1 Instalacja narzędzia

Do analizy użyjemy narzędzia **VTune**, które załadujemy na Aresie przy pomocy komendy:

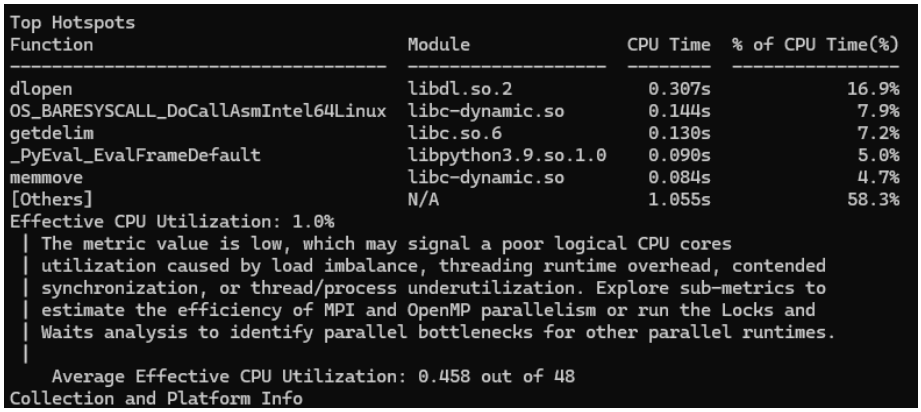
```
module load vtune
```

2 Testy narzędzia

W ramach testów uruchomimy algorytm na czterech rdzeniach przy użyciu narzędzia **VTune**. Algorytm będzie rozwiązywał problem o rozmiarze **16 na 16**. Skrypt uruchamiamy następująco:

```
vtune -collect hotspots -duration 30 -result-dir vtune_results
python lab3.py
```

Po wywołaniu komedy, VTune wyświetli w terminalu:



Rysunek 1: VTune w terminalu

Możemy zauważyć bardzo niską **efektywność wykorzystania CPU**. Prawdopodobnie jest to związane z:

- niewielkim rozmiarem problemu,
- oczekiwaniami VTune'a odnośnie ilości CPU - widzimy, że oprogramowanie oczekuje użycia 48 procesorów,

Zasadne wydaje się zatem znaczne zwiększenie rozmiaru problemu i ilości rdzeni w ramach dalszych eksperymentów. W tym laboratorium mużyjemy **metody Gaussa-Seidla** do rozwiązania zadania.

3 Analiza wydajności algorytmu dla różnych rozmiarów problemów

Rozszerzmy wyżej wspomniane eksperymenty o inne wielkości problemów i dodatkowe rdzenie. Poprawmy także zawarty w nich błąd - rozmiary problemów powinny być kolejnymi **potęgami dwójki** większymi od 2.

3.1 Nowy algorytm

```
import time
import numpy as np
import matplotlib.pyplot as plt
from mpi4py import MPI

def main():
    Lx: float = 10.0
    Ly: float = 10.0
    Nx: int = 64
    Ny: int = 64
```

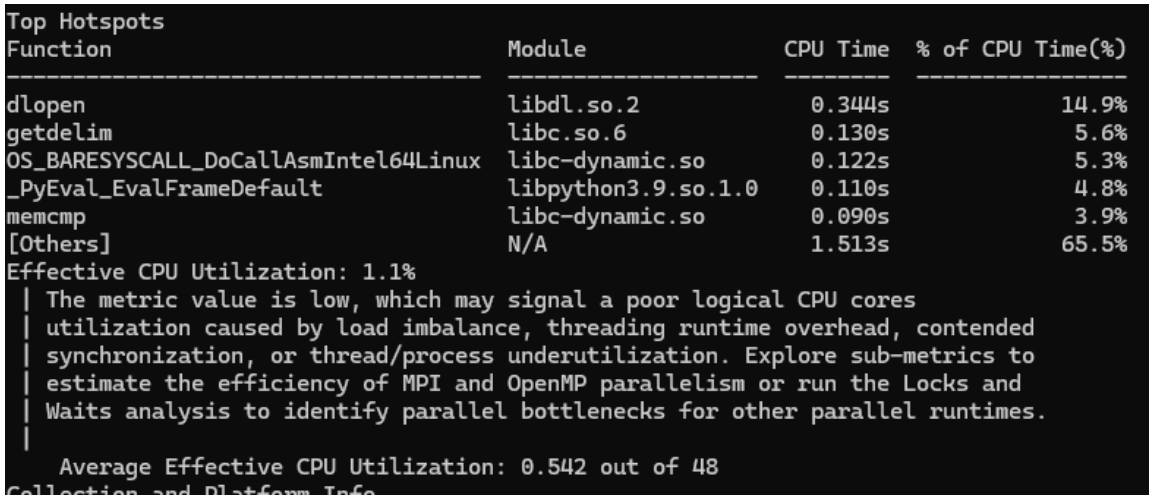
```
    g: float = 1.0
    lambda_: float = 1.0
    omega: float = 1.8

    dx = Lx / (Nx - 1)
    dy = Ly / (Ny - 1)

    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    rows_per_process = Ny // size
    extra_rows = Ny % size
    local_rows = rows_per_process + 1 if rank < extra_rows else rows_per_process

    T_local = np.zeros((local_rows + 2, Nx))
    T_local[0, :] = 0
    T_local[-1, :] = 0

    tolerance: float = 1e-6
    error: float = 1.0

    while error > tolerance:
        error_local = 0.0

        for j in range(1, local_rows + 1):
            for i in range(1, Nx - 1):
                T_new = (T_local[j + 1, i] + T_local[j - 1, i] +
                         T_local[j, i + 1] + T_local[j, i - 1] -
                         (g / lambda_) * (dx ** 2 + dy ** 2)) / 4.0
                T_local[j, i] = (1 - omega) * T_local[j, i] + omega * T_new
                error_local = max(error_local, abs(T_local[j, i] - T_new))

        reqs = []
        if rank > 0:
            reqs.append(comm.Irecv(T_local[0, :], source=rank - 1, tag=0))
            reqs.append(comm.Isend(T_local[1, :], dest=rank - 1, tag=1))
        if rank < size - 1:
            reqs.append(comm.Irecv(T_local[-1, :], source=rank + 1, tag=1))
            reqs.append(comm.Isend(T_local[-2, :], dest=rank + 1, tag=0))

        MPI.Request.Waitall(reqs)

        error = comm.allreduce(error_local, op=MPI.MAX)

    T_local_flat = T_local[1:-1, :].flatten()
```

```
    counts = np.array(comm.gather(len(T_local_flat), root=0))

    if rank == 0:
        T_global_flat = np.empty(sum(counts), dtype=T_local.dtype)
    else:
        T_global_flat = None

    comm.Gatherv(T_local_flat, (T_global_flat, counts), root=0)

    if rank == 0:
        T_global = np.zeros((Ny, Nx))
        row_offset = 0
        for i in range(size):
            local_rows_i = rows_per_process + 1 if i < extra_rows
            else rows_per_process
            T_global[row_offset:row_offset + local_rows_i, :] = \
                T_global_flat[row_offset * Nx:(row_offset
                + local_rows_i) * Nx].reshape(local_rows_i, Nx)
            row_offset += local_rows_i
        return T_global
    return None

if __name__ == "__main__":
    start = time.time()
    T_global = main()
    end = time.time()
    print(f"Execution Time: {(end - start):.7f} seconds")
```

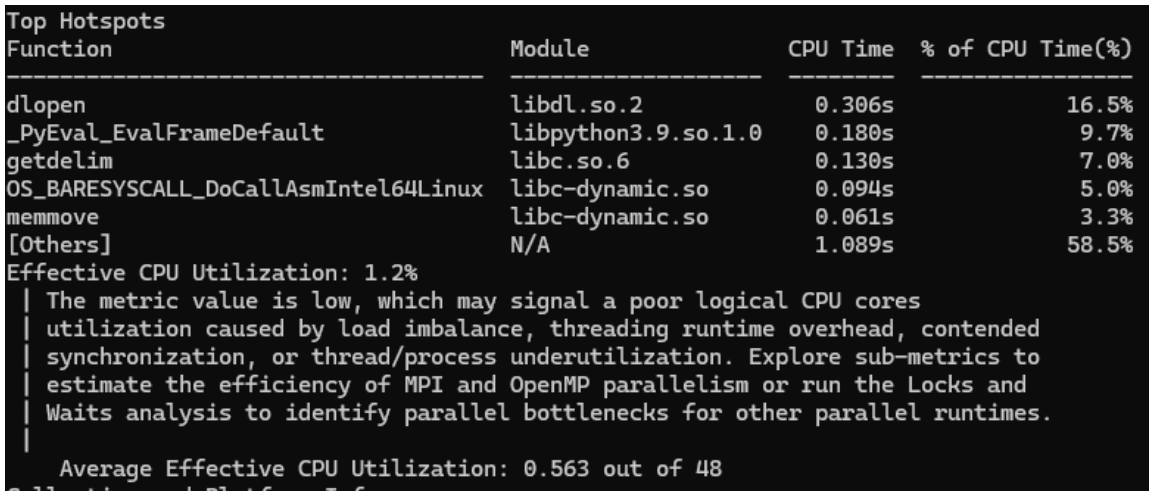
3.2 Rozmiar 32 na 32

Liczba rdzeni	Średni czas wykonania algorytmu (s)
1 (implementacja sekwencyjna)	2.91
2	1.49
4	0.28
8	0.26
16	0.22
32	0.23
48	0.24

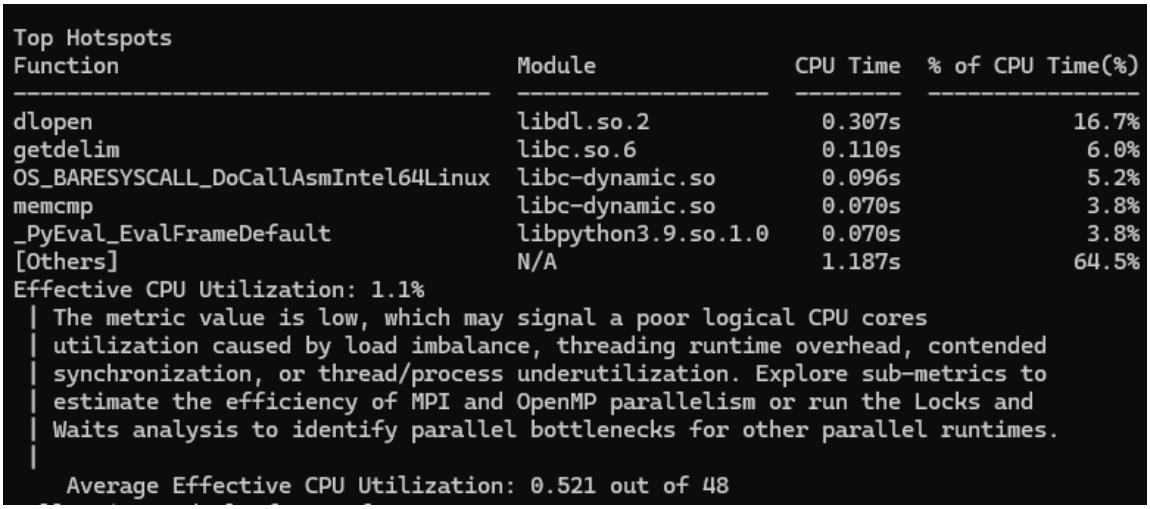
Tabela 1: Średni czas obliczeń dla problemu o rozmiarze 32 na 32.



Rysunek 2: Rozmiar 32 na 32, 4 procesory.

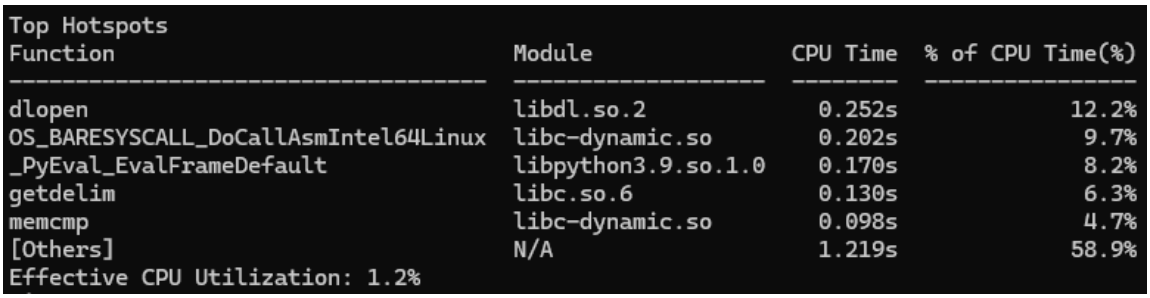


Rysunek 3: Rozmiar 32 na 32, 8 procesorów.



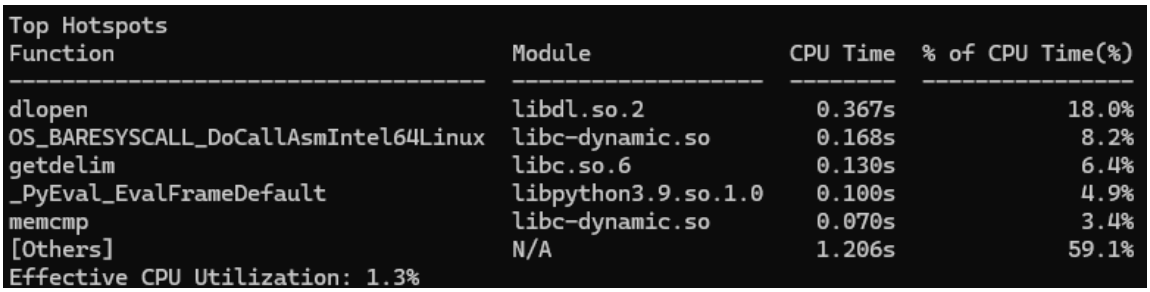
```
Top Hotspots
Function                                Module              CPU Time  % of CPU Time(%)
----------------------------------  ------------------  --------  ----------------
dlopen                                libdl.so.2             0.307s             16.7%
getdelim                              libc.so.6              0.110s              6.0%
OS_BARESYSCALL_DoCallAsmIntel64Linux  libc-dynamic.so        0.096s              5.2%
memcmp                                libc-dynamic.so        0.070s              3.8%
_PyEval_EvalFrameDefault              libpython3.9.so.1.0    0.070s              3.8%
[Others]                              N/A                    1.187s             64.5%
Effective CPU Utilization: 1.1%
  | The metric value is low, which may signal a poor logical CPU cores
  | utilization caused by load imbalance, threading runtime overhead, contended
  | synchronization, or thread/process underutilization. Explore sub-metrics to
  | estimate the efficiency of MPI and OpenMP parallelism or run the Locks and
  | Waits analysis to identify parallel bottlenecks for other parallel runtimes.
  |
    Average Effective CPU Utilization: 0.521 out of 48
```

Rysunek 4: Rozmiar 32 na 32, 16 procesorów.



```
Top Hotspots
Function                                Module              CPU Time  % of CPU Time(%)
----------------------------------  ------------------  --------  ----------------
dlopen                                libdl.so.2             0.252s             12.2%
OS_BARESYSCALL_DoCallAsmIntel64Linux  libc-dynamic.so        0.202s              9.7%
_PyEval_EvalFrameDefault              libpython3.9.so.1.0    0.170s              8.2%
getdelim                              libc.so.6              0.130s              6.3%
memcmp                                libc-dynamic.so        0.098s              4.7%
[Others]                              N/A                    1.219s             58.9%
Effective CPU Utilization: 1.2%
```

Rysunek 5: Rozmiar 32 na 32, 32 procesory.



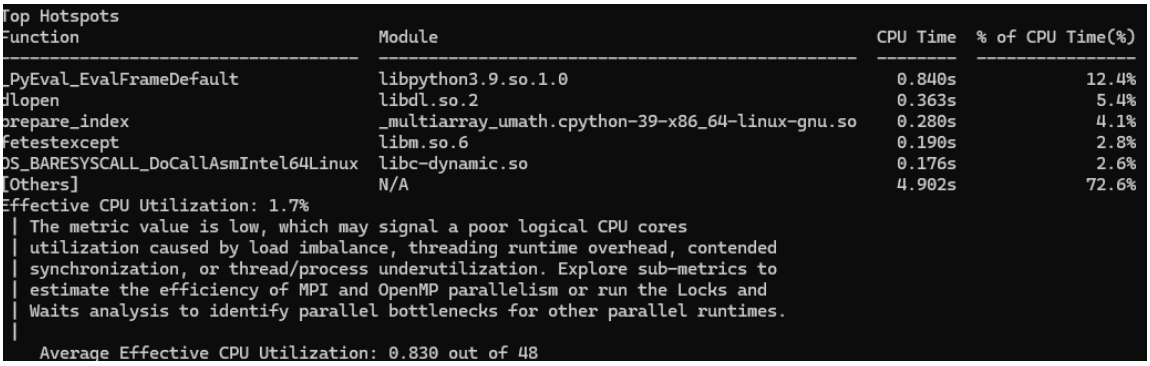
```
Top Hotspots
Function                                Module              CPU Time  % of CPU Time(%)
----------------------------------  ------------------  --------  ----------------
dlopen                                libdl.so.2             0.367s             18.0%
OS_BARESYSCALL_DoCallAsmIntel64Linux  libc-dynamic.so        0.168s              8.2%
getdelim                              libc.so.6              0.130s              6.4%
_PyEval_EvalFrameDefault              libpython3.9.so.1.0    0.100s              4.9%
memcmp                                libc-dynamic.so        0.070s              3.4%
[Others]                              N/A                    1.206s             59.1%
Effective CPU Utilization: 1.3%
```

Rysunek 6: Rozmiar 32 na 32, 48 procesorów.

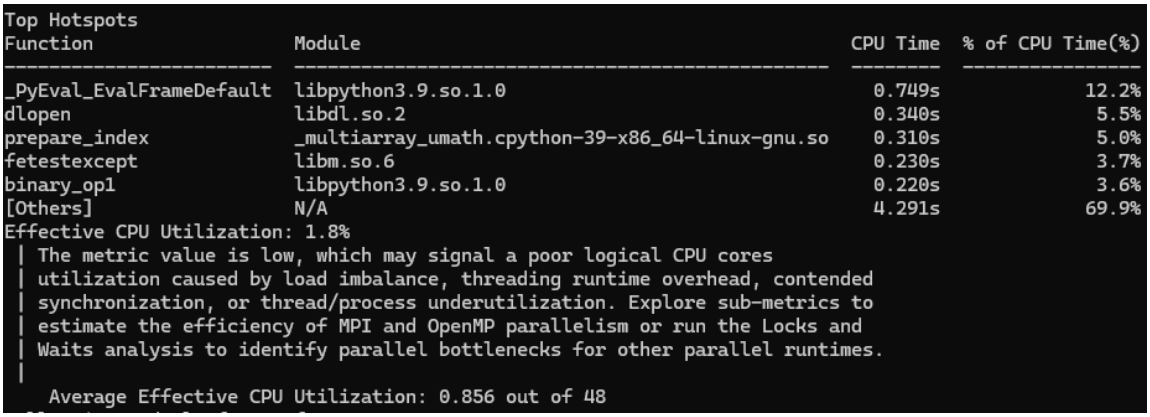
Liczba rdzeni	Średni czas wykonania algorytmu (s)
1 (implementacja sekwencyjna)	38.14
2	22.01
4	4.97
8	4.56
16	4.31
32	4.78
48	45.16

Tabela 2: Średni czas obliczeń dla problemu o rozmiarze 64 na 64.

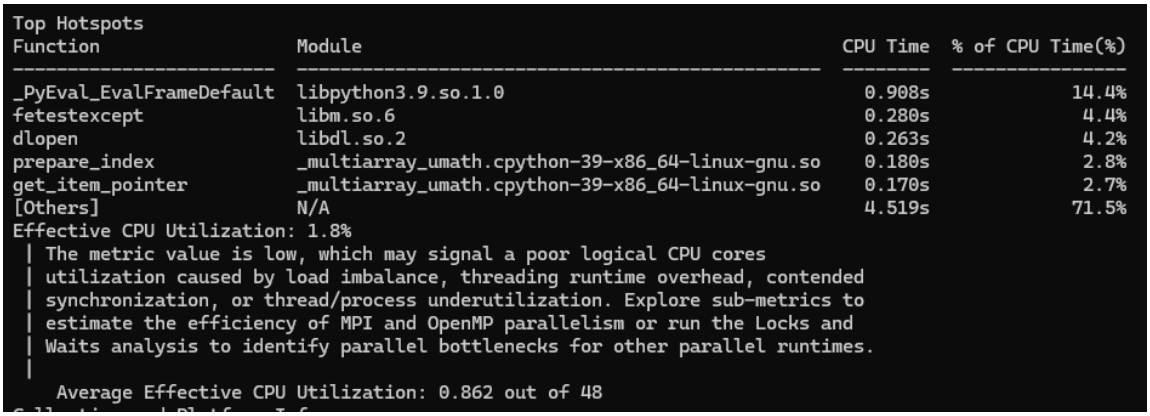
3.3 Rozmiar 64 na 64



Rysunek 7: Rozmiar 64 na 64, 4 procesory.

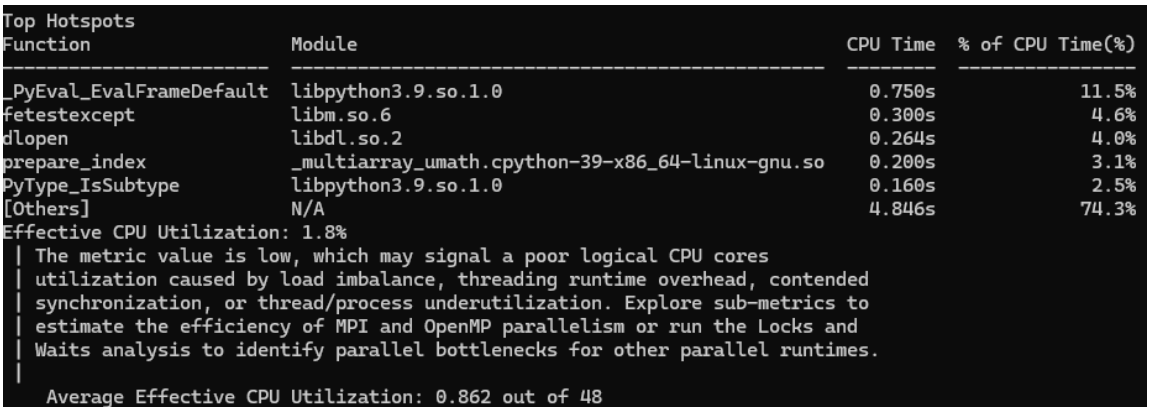


Rysunek 8: Rozmiar 64 na 64, 8 procesorów.



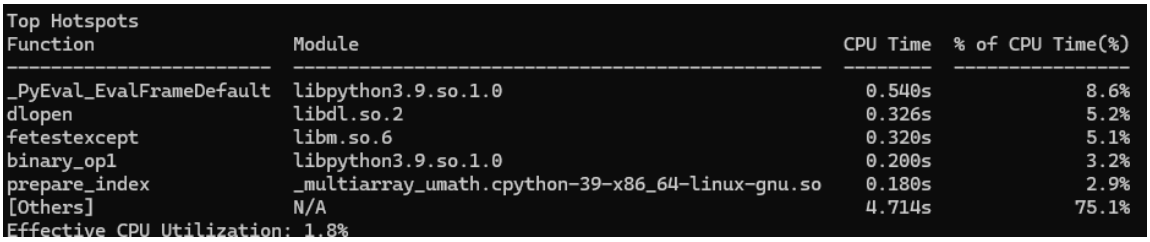
```
Top Hotspots
Function                 Module                                          CPU Time  % of CPU Time(%)
-----------------------  ----------------------------------------------  --------  ----------------
_PyEval_EvalFrameDefault  libpython3.9.so.1.0                              0.908s             14.4%
fetestexcept             libm.so.6                                       0.280s              4.4%
dlopen                   libdl.so.2                                      0.263s              4.2%
prepare_index            _multiarray_umath.cpython-39-x86_64-linux-gnu.so  0.180s              2.8%
get_item_pointer         _multiarray_umath.cpython-39-x86_64-linux-gnu.so  0.170s              2.7%
[Others]                 N/A                                             4.519s             71.5%
Effective CPU Utilization: 1.8%
  | The metric value is low, which may signal a poor logical CPU cores
  | utilization caused by load imbalance, threading runtime overhead, contended
  | synchronization, or thread/process underutilization. Explore sub-metrics to
  | estimate the efficiency of MPI and OpenMP parallelism or run the Locks and
  | Waits analysis to identify parallel bottlenecks for other parallel runtimes.
  |
     Average Effective CPU Utilization: 0.862 out of 48
```

Rysunek 9: Rozmiar 64 na 64, 16 procesorów.



```
Top Hotspots
Function                 Module                                          CPU Time  % of CPU Time(%)
-----------------------  ----------------------------------------------  --------  ----------------
_PyEval_EvalFrameDefault  libpython3.9.so.1.0                              0.750s             11.5%
fetestexcept             libm.so.6                                       0.300s              4.6%
dlopen                   libdl.so.2                                      0.264s              4.0%
prepare_index            _multiarray_umath.cpython-39-x86_64-linux-gnu.so  0.200s              3.1%
PyType_IsSubtype         libpython3.9.so.1.0                             0.160s              2.5%
[Others]                 N/A                                             4.846s             74.3%
Effective CPU Utilization: 1.8%
  | The metric value is low, which may signal a poor logical CPU cores
  | utilization caused by load imbalance, threading runtime overhead, contended
  | synchronization, or thread/process underutilization. Explore sub-metrics to
  | estimate the efficiency of MPI and OpenMP parallelism or run the Locks and
  | Waits analysis to identify parallel bottlenecks for other parallel runtimes.
  |
     Average Effective CPU Utilization: 0.862 out of 48
```

Rysunek 10: Rozmiar 64 na 64, 32 procesory.



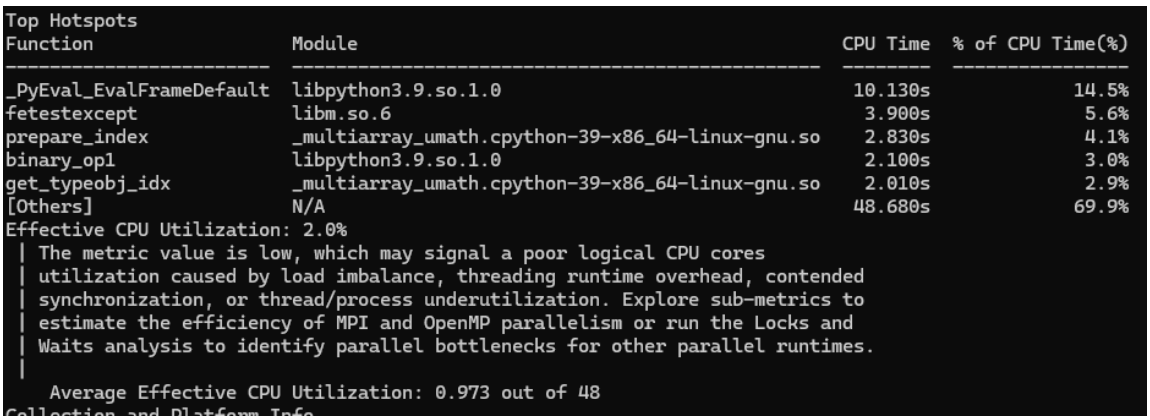
```
Top Hotspots
Function                 Module                                          CPU Time  % of CPU Time(%)
-----------------------  ----------------------------------------------  --------  ----------------
_PyEval_EvalFrameDefault  libpython3.9.so.1.0                              0.540s              8.6%
dlopen                   libdl.so.2                                      0.326s              5.2%
fetestexcept             libm.so.6                                       0.320s              5.1%
binary_op1               libpython3.9.so.1.0                             0.200s              3.2%
prepare_index            _multiarray_umath.cpython-39-x86_64-linux-gnu.so  0.180s              2.9%
[Others]                 N/A                                             4.714s             75.1%
Effective CPU Utilization: 1.8%
```

Rysunek 11: Rozmiar 64 na 64, 48 procesorów.

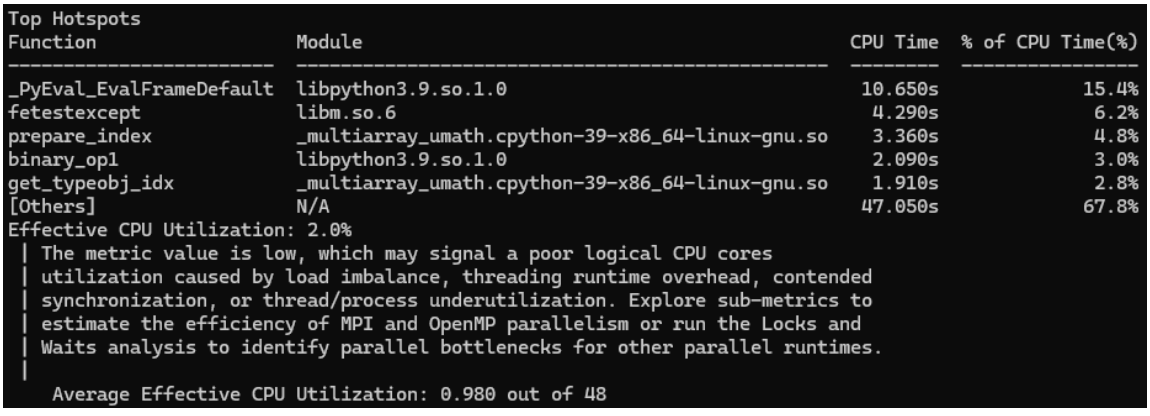
3.4 Rozmiar 128 na 128

Liczba rdzeni	Średni czas wykonania algorytmu (s)
1 (implementacja sekwencyjna)	570.46
2	297.40
4	68.19
8	64.31
16	66.75
32	68.15
48	67.98

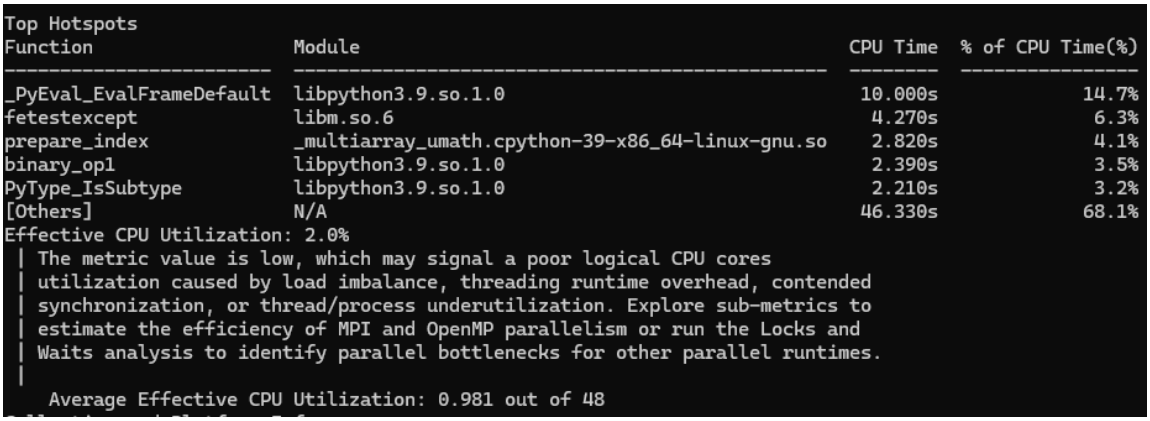
Tabela 3: Średni czas obliczeń dla problemu o rozmiarze 128 na 128.



Rysunek 12: Rozmiar 128 na 128, 4 procesory.

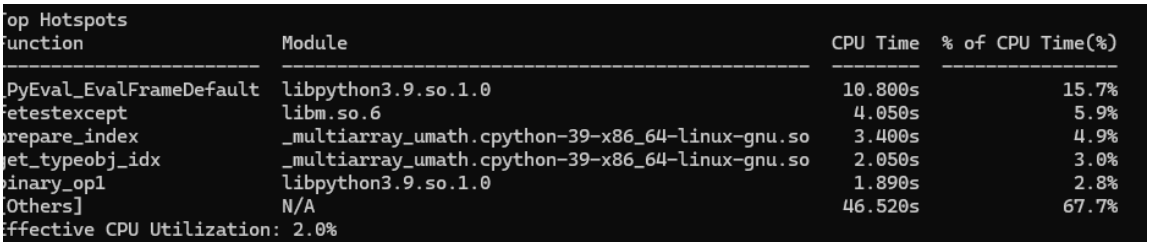


Rysunek 13: Rozmiar 128 na 128, 8 procesorów.



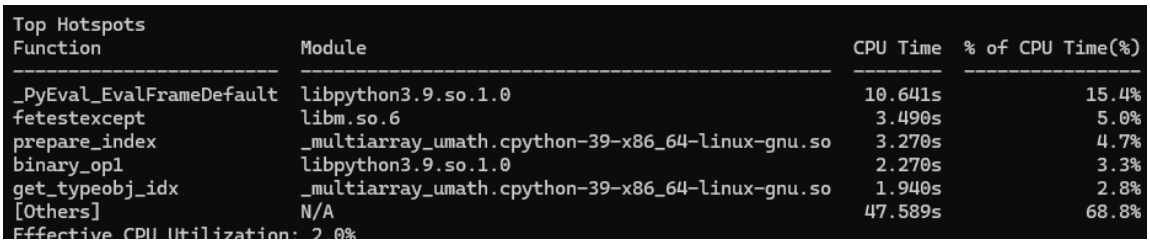
```
Top Hotspots
Function                  Module                                              CPU Time  % of CPU Time(%)
------------------------  --------------------------------------------------  --------  ----------------
_PyEval_EvalFrameDefault  libpython3.9.so.1.0                                 10.000s             14.7%
fetestexcept              libm.so.6                                            4.270s              6.3%
prepare_index             _multiarray_umath.cpython-39-x86_64-linux-gnu.so     2.820s              4.1%
binary_op1                libpython3.9.so.1.0                                  2.390s              3.5%
PyType_IsSubtype          libpython3.9.so.1.0                                  2.210s              3.2%
[Others]                  N/A                                                 46.330s             68.1%
Effective CPU Utilization: 2.0%
  | The metric value is low, which may signal a poor logical CPU cores
  | utilization caused by load imbalance, threading runtime overhead, contended
  | synchronization, or thread/process underutilization. Explore sub-metrics to
  | estimate the efficiency of MPI and OpenMP parallelism or run the Locks and
  | Waits analysis to identify parallel bottlenecks for other parallel runtimes.
  |
    Average Effective CPU Utilization: 0.981 out of 48
```

Rysunek 14: Rozmiar 128 na 128, 16 procesorów.



```
op Hotspots
unction                   Module                                              CPU Time  % of CPU Time(%)
------------------------  --------------------------------------------------  --------  ----------------
PyEval_EvalFrameDefault   libpython3.9.so.1.0                                 10.800s             15.7%
etestexcept               libm.so.6                                            4.050s              5.9%
repare_index              _multiarray_umath.cpython-39-x86_64-linux-gnu.so     3.400s              4.9%
et_typeobj_idx            _multiarray_umath.cpython-39-x86_64-linux-gnu.so     2.050s              3.0%
inary_op1                 libpython3.9.so.1.0                                  1.890s              2.8%
Others]                   N/A                                                 46.520s             67.7%
ffective CPU Utilization: 2.0%
```

Rysunek 15: Rozmiar 128 na 128, 32 procesory.



```
Top Hotspots
Function                  Module                                              CPU Time  % of CPU Time(%)
------------------------  --------------------------------------------------  --------  ----------------
_PyEval_EvalFrameDefault  libpython3.9.so.1.0                                 10.641s             15.4%
fetestexcept              libm.so.6                                            3.490s              5.0%
prepare_index             _multiarray_umath.cpython-39-x86_64-linux-gnu.so     3.270s              4.7%
binary_op1                libpython3.9.so.1.0                                  2.270s              3.3%
get_typeobj_idx           _multiarray_umath.cpython-39-x86_64-linux-gnu.so     1.940s              2.8%
[Others]                  N/A                                                 47.589s             68.8%
Effective CPU Utilization: 2.0%
```

Rysunek 16: Rozmiar 128 na 128, 48 procesorów.

4 Wnioski

Ze względu na ograniczenia dostępu do większych ilości procesorów, udało się rozwiązać jedynie relatywnie małe problemy. Efektywności użycia CPU jest niska, ale widzimy, że wzrasta razem ze zwiększaniem rozmiaru problemu.